

|                                                                                                                                                                                                                                   |       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Computer Systems                                                                                                                                                                                                                  | • BA4 |
| 1 Lecture 5 - Network & File Syscalls                                                                                                                                                                                             |       |
| Permission bits                                                                                                                                                                                                                   |       |
| Am I the <b>owner</b> ? → use USR bits.<br>Am I in the <b>group</b> ? → use GRP bits.<br>Otherwise → use OTH bits.<br><i>Only the first matching category applies — even if another gives more rights.</i>                        |       |
| Syscall success conditions                                                                                                                                                                                                        |       |
| open(O_RDONLY)    r on file<br>open(O_WRONLY)   w on file<br>open(O_RDWR)    r+w on file<br>open(O_CREAT)    w+x on dir<br>read(fd,...)    fd open, mode=read<br>write(fd,...)    fd open, mode=write<br>close(fd)        fd open |       |
| <b>Every dir</b> in the path needs <b>x</b> to be traversed.<br>Any syscall on a <b>closed</b> fd fails immediately.                                                                                                              |       |

|                                                                                     |
|-------------------------------------------------------------------------------------|
| 8_* flags                                                                           |
| Only used with O_CREAT to set the new file's permissions. <b>Ignored</b> otherwise. |

|                                                      |
|------------------------------------------------------|
| 2 Lecture 8 - Forwarding and IP                      |
| Forwarded using the <b>longest matching prefix</b> . |

|                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Broadcast IP address (+1)                                                                                                                                      |
| The <b>biggest IP address</b> an IP subnet. If a packet has this destination, it's destined to <b>all the end-systems</b> and <b>routers</b> in the IP subnet. |

|                                                               |
|---------------------------------------------------------------|
| Global reachability                                           |
| Every end-system must be reachable from any other end-system. |

|                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NAT: Layering violation (but it works)                                                                                                                                                                                                                                                                                                                        |
| A router becomes a NAT only when it's configured to translate between private and public addresses. <ul style="list-style-type: none"> <li>Requires <b>per-connection state</b> at the border routers of the private IP subnets.</li> <li>An end-system inside a private IP subnet is <b>un-reachable</b> from the outside world. [trick possible]</li> </ul> |

|                             |
|-----------------------------|
| 3 Lecture 9 - Routing & BGP |
|-----------------------------|

|                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dijkstra's (Link state: converges faster)                                                                                                                           |
| 1. At each step, consider a new router starting from "closest" neighbor<br>2. Check whether current paths can be improved<br>3. End when no improvement is possible |

|                                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bellman-Ford (Distance vector: less bandwithh)                                                                                                                                  |
| 1. All neighbors exchange information<br>2. Each router checks whether it can improve current paths by leveraging the new information<br>3. End when no improvement is possible |

|                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Internet routing challenges                                                                                                                                         |
| <ul style="list-style-type: none"> <li><b>Admin:</b> An ISP may want to hide its link costs</li> <li><b>Scale:</b> Too many IP subnets may cause trouble</li> </ul> |

|                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------|
| Border Gateway Protocol (BGP)                                                                                                                |
| Inter-AS Routing : Not always the SP. Bellman                                                                                                |
| 1. All neighbors exchange reachable prefixes<br>2. Each router checks whether it can improve paths<br>3. End when no improvement is possible |

|                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Internet routing solutions                                                                                                                                                                                                                                           |
| <ul style="list-style-type: none"> <li>Router ↔ with <b>local</b> router</li> <li>Border router ↔ with <b>external neighbor</b></li> <li>Router keeps <b>forwarding state</b> for (1) <b>local IP subnets</b> and (2) <b>foreign aggregate IP prefixe</b></li> </ul> |

|                                      |
|--------------------------------------|
| 4 Lecture 10 - Link layer & Ethernet |
|--------------------------------------|

|                                                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Address Resolution Protocol (ARP)                                                                                                                                                                       |
| Resolves an IP address to a MAC address (to build the L2 frame) via <i>broadcast</i> + <i>caching</i> .<br>Naïve broadcast may cause forwarding loops ⇒ use a <b>spanning tree</b> to eliminate cycles. |

|                             |
|-----------------------------|
| 5 Lecture 11 - File Systems |
|-----------------------------|

|                                                                                          |
|------------------------------------------------------------------------------------------|
| Persistence                                                                              |
| State that <b>outlives the process that created it</b> (unlike DRAM, which is volatile). |

|                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------|
| File System                                                                                                              |
| Abstracts persistent blocks into: <b>Name, Organize, Share, Protect</b> . Built on <b>files</b> and <b>directories</b> . |

|                                                                                                     |
|-----------------------------------------------------------------------------------------------------|
| File                                                                                                |
| Named array of bytes (untyped) with <b>data</b> + <b>meta-data</b> (size, owner, permissions, ...). |

|                                                                                              |
|----------------------------------------------------------------------------------------------|
| Inode                                                                                        |
| Fixed-size on-disk record — one per file: <b>metadata</b> + <b>pointers to data blocks</b> . |

|                                      |
|--------------------------------------|
| Inode Table                          |
| Fixed array indexed by inode number. |

|                                                         |
|---------------------------------------------------------|
| Directory (special file)                                |
| Array of { <b>filename, inode</b> } pairs per partition |

|                                                                                |
|--------------------------------------------------------------------------------|
| File Descriptor Table                                                          |
| Per-process. fd → ( <b>inode, mode, offset</b> ). 0=STDIN, 1=STDOUT, 2=STDERR. |

|                                      |
|--------------------------------------|
| 6 Lecture 12 - Processes and Threads |
|--------------------------------------|

|                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Process                                                                                                                                         |
| An instance of a running program. Has its own <b>isolated memory space</b> (code, data, heap, stack) and OS resources (file descriptors, etc.). |

|                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Thread                                                                                                                                                        |
| A unit of execution <b>within</b> a process. Threads share the process's memory and resources, but each has its own <b>stack</b> and <b>program counter</b> . |

|                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Key properties                                                                                                                                                                                                         |
| <ul style="list-style-type: none"> <li><b>Identifier:</b> unique id for processes and threads</li> <li>Each Thread/Process has the <b>illusion</b> that is it the only one occupying the CPU/Memory (resp.)</li> </ul> |

|                                |
|--------------------------------|
| 7 Lecture 13 - Sharing the CPU |
|--------------------------------|

|                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CPU mode & instructions                                                                                                                                                                                                        |
| CPU stores a privilege level: <b>Kernel mode</b> (high) or <b>User mode</b> (low). <ul style="list-style-type: none"> <li><b>Non-privileged:</b> executable anytime</li> <li><b>Privileged:</b> only in kernel mode</li> </ul> |

|                                                |
|------------------------------------------------|
| Limited direct execution (invoking the kernel) |
| Syscall / Exception / Interrupt                |

|                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Process lifecycle                                                                                                                                                                                                                                                                  |
| <ul style="list-style-type: none"> <li><b>fork:</b> creates a <b>child</b> process (copy of parent)</li> <li><b>exec:</b> replaces process image with a new program</li> <li><b>exit:</b> terminates the process</li> <li><b>wait:</b> parent waits for child to finish</li> </ul> |

|                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Type of system processes                                                                                                                                                                                                                               |
| <ul style="list-style-type: none"> <li><b>Init:</b> first process after boot</li> <li><b>Idle:</b> runs when nothing else can</li> <li><b>Shell:</b> spawns new process on command</li> <li><b>GUI manager:</b> spawns new process on click</li> </ul> |

|                                    |
|------------------------------------|
| 8 Lecture 14 - Memory & Loose Ends |
|------------------------------------|

|                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| End of a process                                                                                                                                                                           |
| <b>exit()</b> syscall: kernel destroys memory image but keeps <b>exit status</b> ⇒ process becomes a <b>zombie</b> until parent <b>reaps</b> it via <b>wait()</b> to read the exit status. |

|                               |
|-------------------------------|
| 9 Lecture 15 - CPU Scheduling |
|-------------------------------|

|                                                                                               |
|-----------------------------------------------------------------------------------------------|
| Scheduling Metrics                                                                            |
| <b>Turnaround time</b> = completion - arrival.<br><b>Response time</b> = first run - arrival. |

|                                                                                                   |
|---------------------------------------------------------------------------------------------------|
| FIFO                                                                                              |
| Run jobs in arrival order. Suffers from <b>convoy effect</b> : short jobs stuck behind long ones. |

|                                                                                               |
|-----------------------------------------------------------------------------------------------|
| SJF (Shortest Job First)                                                                      |
| Run shortest job first. Optimal turnaround if all jobs arrive simultaneously. Non-preemptive. |

|                                                                                   |
|-----------------------------------------------------------------------------------|
| STCF (Shortest Time-to-Completion First)                                          |
| Preemptive SJF: preempt if a new job has less remaining time. Optimal turnaround. |

|                                                                                                 |
|-------------------------------------------------------------------------------------------------|
| Round Robin (RR)                                                                                |
| Each job runs for a fixed <b>quantum</b> , then preempted. Good response time, poor turnaround. |

|                                                                                             |
|---------------------------------------------------------------------------------------------|
| Tradeoff                                                                                    |
| SJF/STCF optimizes <b>turnaround</b> . RR optimizes <b>response time</b> . Hard to do both. |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MLFQ (Multi-Level Feedback Queue)                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Multiple queues; lower priority = longer time slice. <ul style="list-style-type: none"> <li><b>R1:</b> priority(A) &gt; priority(B) ⇒ A runs</li> <li><b>R2:</b> priority(A) = priority(B) ⇒ RR</li> <li><b>R3:</b> new job starts at <b>top</b> priority</li> <li><b>R4:</b> uses full slice ⇒ <b>demoted</b>; blocks early ⇒ keeps priority</li> <li><b>R5:</b> periodically move all jobs to top (<b>priority boost</b>) — avoids starvation</li> </ul> |

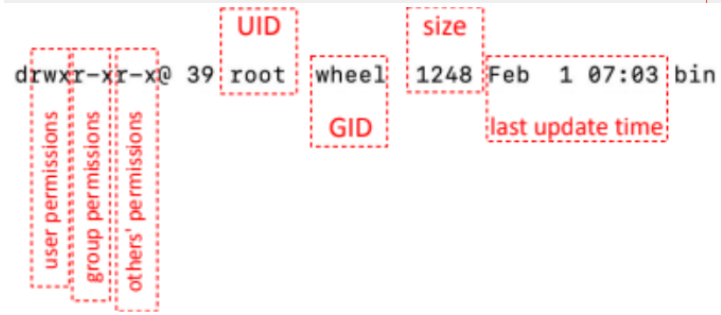
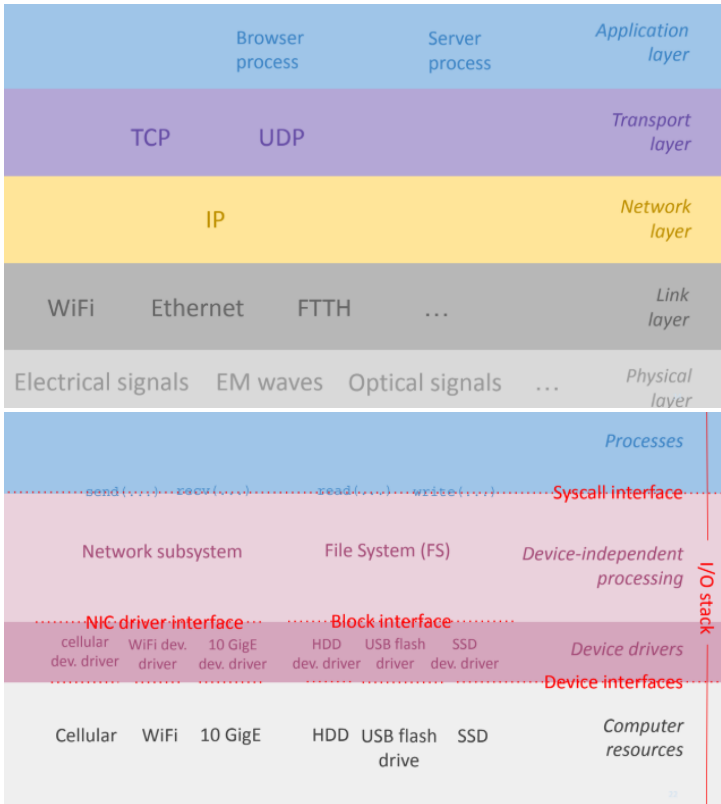
|                                |
|--------------------------------|
| 10 Lecture 17/18 - Concurrency |
|--------------------------------|

|                                                                                                      |
|------------------------------------------------------------------------------------------------------|
| <b>Concurrency:</b> how should we manage tasks<br><b>Parallism:</b> how to execute tasks in parallel |
|------------------------------------------------------------------------------------------------------|

|                                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Race condition & Data race                                                                                                                                                                  |
| <ul style="list-style-type: none"> <li><b>Race cond.:</b> behavior depends on thread timing</li> <li><b>Data race:</b> unsynchronized concurrent access to shared writable state</li> </ul> |

|                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Critical section                                                                                                                                |
| Code that will be executed by the threads. Must enforce: <b>atomicity</b> (uninterruptible) and <b>mutual exclusion</b> (one thread at a time). |

|                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synchronization primitives                                                                                                                                                                                    |
| <ul style="list-style-type: none"> <li><b>Lock</b> → prevent concurrent access</li> <li><b>Condition var.</b> → wait until a condition holds</li> <li><b>Semaphore</b> → count available resources</li> </ul> |



| open() flags / permission mode |                                 |
|--------------------------------|---------------------------------|
| O_CREAT                        | create file if it doesn't exist |
| O_EXCL                         | fail if file already exists     |
| O_APPEND                       | always write at end of file     |
| O_TRUNC                        | truncate file on open           |
| O_RDONLY                       | read only                       |
| O_WRONLY                       | write only                      |
| S_IRUSR                        | read by owner                   |
| S_IWUSR                        | write by owner                  |
| S_IXUSR                        | execute by owner                |
| S_IRGRP                        | read by group                   |
| S_IROTH                        | read by others                  |
| 0                              | no permissions                  |

```

INIT-SINGLE-SOURCE(G, s)
    s.d = 0
    for v in G.V: v.d = inf; v.pi = NIL

RELAX(u, v, w)
    if v.d > u.d + w(u, v)
        v.d = u.d + w(u, v); v.pi = u

BELLMAN-FORD(G, w, s)
    INIT-SINGLE-SOURCE(G, s)
    for i = 1 to |G.V| - 1
        for (u, v) in G.E:
            RELAX(u, v, w)

DIJKSTRA(G, w, s)
    INIT-SINGLE-SOURCE(G, s)
    S = {}
    Q = G.V // min-priority queue (sorted by d(v))
    while Q != {}:
        u = EXTRACT-MIN(Q)
        S = S U {u}
        for each vertex v in G.Adj[u]:
            RELAX(u, v, w)
  
```

Bellman-Ford applied on router Z:

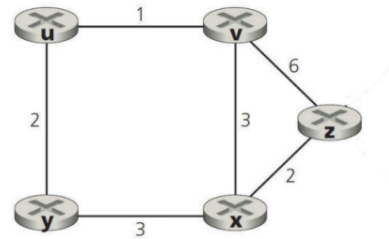


Figure 5: Network topology.

Initially (at step 0), node z has the following view of the network:

|      |   | To |   |   |   |   |
|------|---|----|---|---|---|---|
|      |   | u  | v | x | y | z |
| From | v | ∞  | ∞ | ∞ | ∞ | ∞ |
|      | x | ∞  | ∞ | ∞ | ∞ | ∞ |
|      | z | ∞  | 6 | 2 | ∞ | 0 |

At step 1:

|      |   | To |   |   |   |   |
|------|---|----|---|---|---|---|
|      |   | u  | v | x | y | z |
| From | v | 1  | 0 | 3 | ∞ | 6 |
|      | x | ∞  | 3 | 0 | 3 | 2 |
|      | z | 7  | 5 | 2 | 5 | 0 |

At step 2:

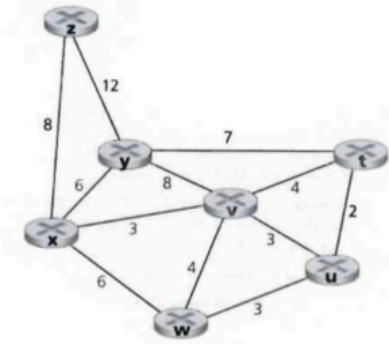
|      |   | To |   |   |   |   |
|------|---|----|---|---|---|---|
|      |   | u  | v | x | y | z |
| From | v | 1  | 0 | 3 | 3 | 5 |
|      | x | 4  | 3 | 0 | 3 | 2 |
|      | z | 6  | 5 | 2 | 5 | 0 |

At step 3:

|      |   | To |   |   |   |   |
|------|---|----|---|---|---|---|
|      |   | u  | v | x | y | z |
| From | v | 1  | 0 | 3 | 3 | 5 |
|      | x | 4  | 3 | 0 | 3 | 2 |
|      | z | 6  | 5 | 2 | 5 | 0 |

We see that from step 2 to step 3 the cost tables did not change, indicating that the algorithm has converged.

Dijkstra for least cost path from x, v and t to the rest.



The least cost path from routers x, v, and t to all the other routers is displayed in the next table along with the execution steps of the link-state algorithm.

For each router i, C(i) stands for cost to i, and p(i) stands for predecessor to i.

1. Least-cost path from x to all network nodes.

| step | nodes visited | C(t),p(t) | C(u),p(u) | C(v),p(v) | C(w),p(w) | C(y),p(y) | C(z),p(z) |
|------|---------------|-----------|-----------|-----------|-----------|-----------|-----------|
| 0    | x             | ∞         | ∞         | 3,x       | 6,x       | 6,x       | 8,x       |
| 1    | x,v           | 7,v       | 6,v       | 3,x       | 6,x       | 6,x       | 8,x       |
| 2    | x,v,u         | 7,v       | 6,v       | 3,x       | 6,x       | 6,x       | 8,x       |
| 3    | x,v,u,w       | 7,v       | 6,v       | 3,x       | 6,x       | 6,x       | 8,x       |
| 4    | x,v,u,w,y     | 7,v       | 6,v       | 3,x       | 6,x       | 6,x       | 8,x       |
| 5    | x,v,u,w,y,t   | 7,v       | 6,v       | 3,x       | 6,x       | 6,x       | 8,x       |
| 6    | x,v,u,w,y,t,z | 7,v       | 6,v       | 3,x       | 6,x       | 6,x       | 8,x       |

2. Least-cost path from v to all network nodes.

| step | nodes visited | C(t),p(t) | C(u),p(u) | C(w),p(w) | C(x),p(x) | C(y),p(y) | C(z),p(z) |
|------|---------------|-----------|-----------|-----------|-----------|-----------|-----------|
| 0    | v             | 4,v       | 3,v       | 4,v       | 3,v       | 8,v       | ∞         |
| 1    | v,x           | 4,v       | 3,v       | 4,v       | 3,v       | 8,v       | 11,x      |
| 2    | v,x,u         | 4,v       | 3,v       | 4,v       | 3,v       | 8,v       | 11,x      |
| 3    | v,x,u,t       | 4,v       | 3,v       | 4,v       | 3,v       | 8,v       | 11,x      |
| 4    | v,x,u,t,w     | 4,v       | 3,v       | 4,v       | 3,v       | 8,v       | 11,x      |
| 5    | v,x,u,t,w,y   | 4,v       | 3,v       | 4,v       | 3,v       | 8,v       | 11,x      |
| 6    | v,x,u,t,w,y,z | 4,v       | 3,v       | 4,v       | 3,v       | 8,v       | 11,x      |

3. Least-cost path from t to all network nodes.

| step | nodes visited | C(u),p(u) | C(v),p(v) | C(w),p(w) | C(x),p(x) | C(y),p(y) | C(z),p(z) |
|------|---------------|-----------|-----------|-----------|-----------|-----------|-----------|
| 0    | t             | 2,t       | 4,t       | ∞         | ∞         | 7,t       | ∞         |
| 1    | t,u           | 2,t       | 4,t       | 5,u       | ∞         | 7,t       | ∞         |
| 2    | t,u,v         | 2,t       | 4,t       | 5,u       | 7,v       | 7,t       | ∞         |
| 3    | t,u,v,w       | 2,t       | 4,t       | 5,u       | 7,v       | 7,t       | ∞         |
| 4    | t,u,v,w,x     | 2,t       | 4,t       | 5,u       | 7,v       | 7,t       | 15,x      |
| 5    | t,u,v,w,x,y   | 2,t       | 4,t       | 5,u       | 7,v       | 7,t       | 15,x      |
| 6    | t,u,v,w,x,y,z | 2,t       | 4,t       | 5,u       | 7,v       | 7,t       | 15,x      |

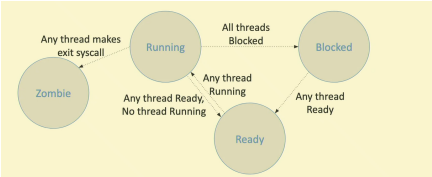
Powers of ten

- 1 KB (kilobyte) =  $10^3$  = 1 000 bytes
- 1 MB (megabyte) =  $10^6$  = 1 000 000 bytes
- 1 GB (gigabyte) =  $10^9$  = 1 000 000 000 bytes

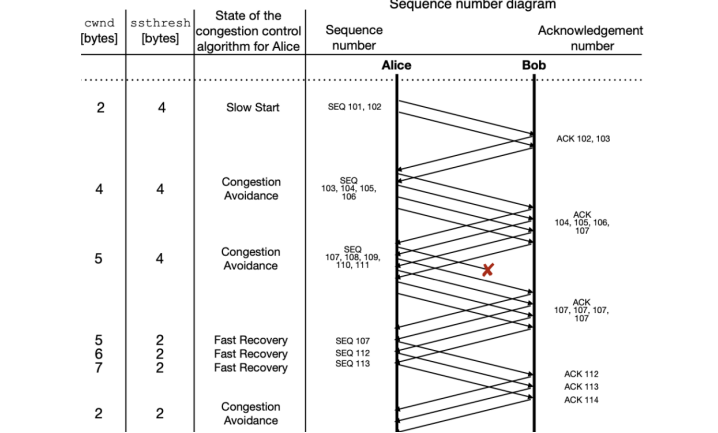
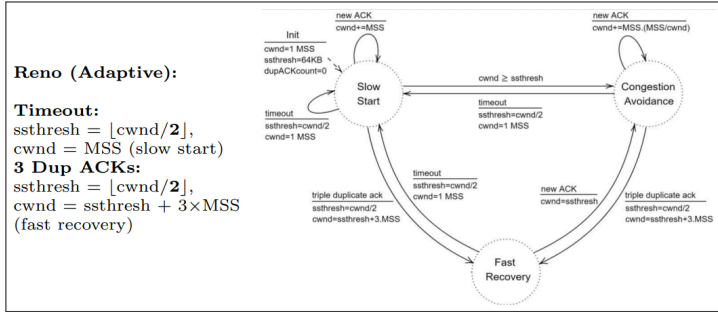
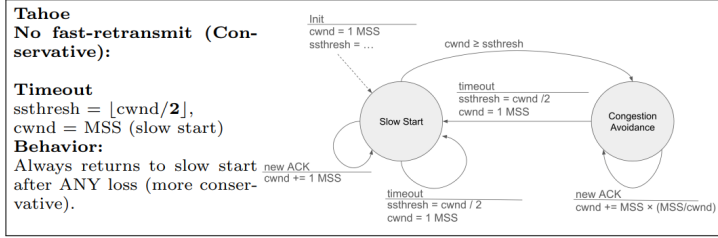
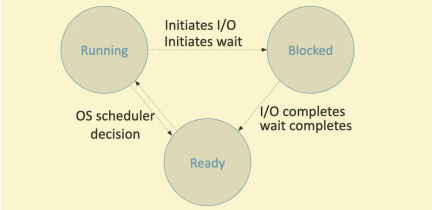
Powers of two

- 1 KiB (kibibyte) =  $2^{10}$  = 1 024 bytes
- 1 MiB (mebibyte) =  $2^{20}$  = 1 048 576 bytes
- 1 GiB (gibibyte) =  $2^{30}$  = 1 073 741 824 bytes

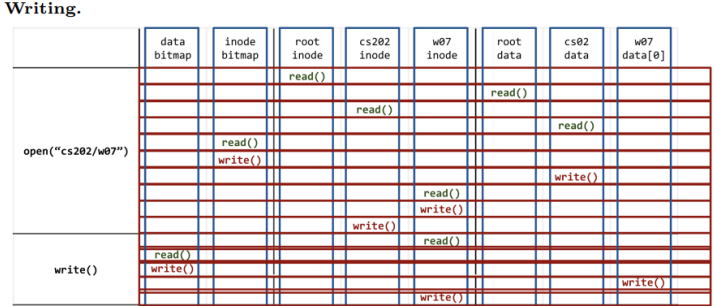
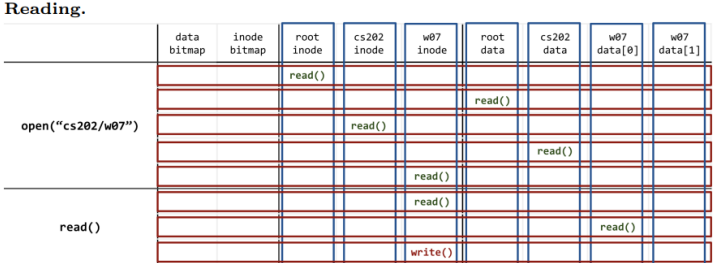
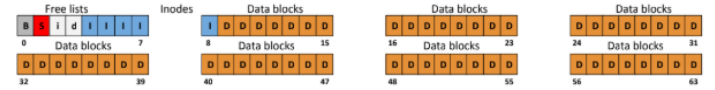
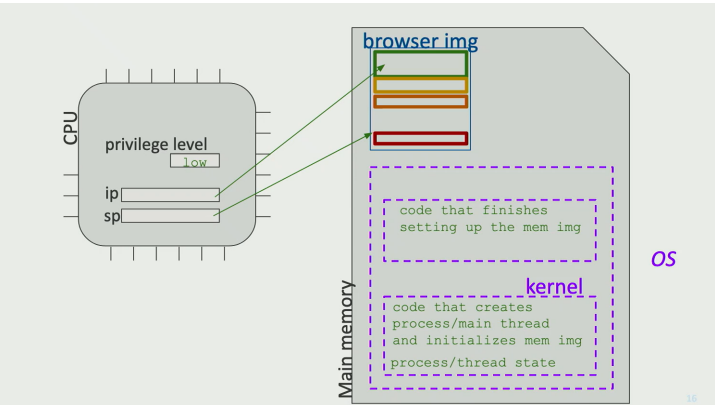
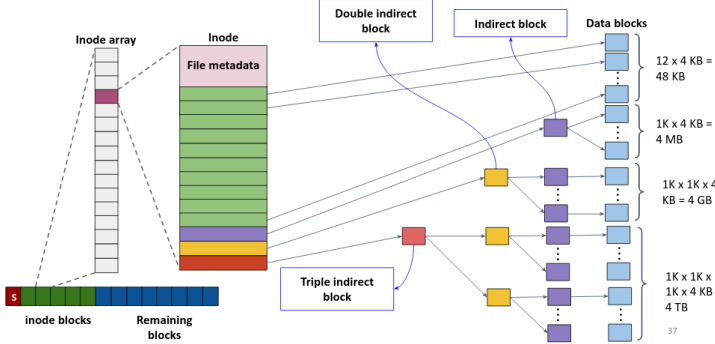
Process state diagram



Thread state diagram



## 4. Multi-level indexing



## Web and DNS

All web client processes keep caches with the most recently retrieved web objects. All DNS client and DNS server processes keep caches with the most recently retrieved DNS records (e.g., DNS name → IP address mappings). If the TTL of a retrieved web object or DNS record is not explicitly stated, assume that it is 1 hour. Web processes communicate over TCP. DNS processes communicate over UDP.

## TCP

When a TCP-related question does not mention the receiver window at all, assume that the receiver window is very large and it does not affect the rate at which data is exchanged. When you are given a scenario where Processes A and B exchange data over TCP, assume that they use the same TCP connection for all the data, unless you are explicitly told otherwise. When you are given a scenario where client Process A establishes a TCP connection with server Process B and A sends a request to B; assume that the second segment sent by A to B already carries bytes of data and has SEQ number 1. When computing the amount of time it takes for a data segment to go from Alice to Bob plus the time it takes for an ACK to go from Bob to Alice, when there is no other traffic present (so, no queuing delays): Do take into account the transmission delay experienced by the data segment, but not for the ACK.

## Packet Switch Behavior

All packet switches are store-and-forward packet switches. All packet switches have infinite queues. All packet switches introduce 0 processing delays. All packet switches behave as follows: There is always one queue per link and it stores packets that are going out that link. When a packet arrives, the switch immediately examines the packet, determines the correct outgoing link, and stores the packet in the queue that feeds that link. If two packets arrive at a switch and are put in the same queue at the same time, the order in which they are put in the queue is random. Assume the order that makes the solution simpler. All queues are served following a FIFO (first in first out) policy.

## Insignificant Headers & Transmission delays

If a problem tells you that “lower-layer headers are insignificant”: If you need to compute the transmission delay of a packet, and that packet does not carry any application-layer content (e.g., it is a TCP segment that carries an ACK but no data), then this packet experiences transmission delay 0.

## IP subnets & prefixes

Each network interface of a router has an IP address (and, of course, a MAC address). Network interfaces of L2 switches have no IP or MAC addresses. Each IP subnet has a broadcast IP address (which is the highest IP address in the subnet), but NO network IP address. Each end-system knows the IP address of its default gateway.

## File access

We don't expect you to reason about bitmap accesses. We do expect you to reason about inode and data block accesses. Reading a file does not trigger a write to the file's inode to update the read access time. Closing a file does not trigger any block access. There exists a file system (FS) cache (which is the same thing as a block cache), unless you are explicitly told that it does not exist. The FS/block cache has infinite size unless you are explicitly told otherwise. The FS/block cache is write-through. I.e., when a process makes a write syscall to write N bytes to disk, this causes the FS/block cache to be updated, but it also triggers as many block accesses as necessary to write the N bytes to disk (and to update the inode of the file that's being written). Differently said: The presence of the FS/block cache may remove the need for some read block accesses, but it cannot remove the need for write block accesses.

## Scheduling

If a thread terminates or blocks after running for a fraction of a time tick (e.g., after 0.99 time ticks), assume that “nothing” happens during the rest of the current time tick. Differently said: Assume that the OS scheduler can only run at integer time-tick values, e.g., time tick 0, time tick 1, etc.